Building data course — Week 2 — lookups, joins and conditions

sparql-playground / lessons

Generated for print and offline reading

Week 2 — lookups, joins and conditions

Five daily lessons. Run the SQL in DB Browser for SQLite against the API's own database, `var/timeseries.sqlite` (build it with `uv run bldgts bootstrap`). Lines like `-- expect: 288` are the answer you should see. Keep notes under the ignored `student-work/` folder.

Contents

- Day 6 — A point list table (`lab_point`), and why it's a stand-in for the RDF model
- Day 7 — JOIN: readings by role and equipment instead of UUID

- Day 8 — Self-joins: static pressure *while the fan is running*
- Day 9 — CASE and conditional aggregation: run hours, percent occupied
- Day 10 — Time buckets and views: hourly averages (what Timescale's `time_bucket` does)

Day 6 — A point list table (`lab_point`), and why it's a stand-in for the RDF model

Goal: get the point list into the database, so queries can say "duct static pressure" instead of a UUID.

Read

The UUID → meaning mapping lives in `model/points/*.csv`, one file per AHU. Today you load both files into one table, `lab_point`. From now on you'll **write** to the database, so:

1. In DB Browser, **Close Database**, then open `var/timeseries.sqlite` normally (not read-only).

2. Close it again before running the command below, because SQLite lets only one writer at a time.

3. In a terminal, in the repo:

```
uv run python lessons/course.py load-
        points
```

4. Reopen the database. A new table, `lab_point`, has one row per point:

Column	Example
timeseries_id	52422584-…
point_id	AHU_1_DA_P
equipment_id	AHU_1
owner_id	AHU_1_Supply_Fan
brick_class	Supply_Air_Static_Pressure_Ser
unit	IN_H2O
openfdd_role	duct-static-pressure
source_column	da_p_inwc

(DB Browser can also do this itself: **File → Import → Table from CSV file**. The script just adds `equipment_id`, which isn't a column in the files.)

Why this table is a compromise. Look at `owner_id`: the supply fan status belongs to `AHU_1_Supply_Fan`, which is part of `AHU_1`. Zone temperatures belong to `Zone_VAV_107`, which AHU_1 feeds. A flat table holds *one* "belongs to" column. A building is a **graph**: fans are part of AHUs, AHUs feed zones, sensors are points of equipment. You'll be tempted to write `owner_id LIKE 'AHU_1%'`. That works here by luck of the naming and breaks on the next site. The RDF model stores those relationships explicitly (`brick:isPartOf`, `brick:feeds`, `brick:isPointOf`), and Units 3–4 query them with SPARQL. For three weeks of SQL, this table is enough.

Run

```
SELECT count(*), count(openfdd_role),
         count(DISTINCT brick_class) FROM
         lab_point;
-- expect: 118 | 26 | 30
```

`count(openfdd_role)` skips NULLs, so 26 of the 118 points have a fault-rule role: 13 per AHU.

```sql
SELECT equipment_id, count(*) AS points
FROM lab_point
GROUP BY equipment_id
ORDER BY equipment_id;
-- expect: AHU_1 | 83


-- The 13 roles on AHU_1
SELECT openfdd_role, point_id, unit,
         timeseries_id
FROM lab_point
WHERE equipment_id = 'AHU_1' AND
         openfdd_role IS NOT NULL
ORDER BY openfdd_role;
-- expect rows: 13
```

`IS NOT NULL`, never `!= NULL`: any comparison with NULL is unknown, so `= NULL` and `!= NULL` match nothing.

Try it

1. What is AHU_2's duct static pressure UUID?
2. Why does AHU_1 have 83 points but AHU_2 only 35? Look at `brick_class`.

Answers

```sql
SELECT timeseries_id FROM lab_point
WHERE equipment_id = 'AHU_2' AND
         openfdd_role = 'duct-static-
         pressure';
-- expect: f8a0d594-2fff-567a-9853-
         71ba66233eba
```

```
SELECT equipment_id, count(*) FROM
         lab_point
WHERE brick_class =
         'Zone_Air_Temperature_Sensor'
GROUP BY equipment_id;
-- expect: AHU_1 | 47
```

AHU_1's export included the zone temperatures of the VAV boxes it serves (47 of them). AHU_2's didn't. Same kind of building, different export: a point list tells you what was **exported**, not what exists.

Why it matters

`lab_point` is a lookup table: id → attributes. In week 5 you'll build the same thing as a dict in Python, then as a set of triples `(point, hasUnit, IN_H2O)`. That's the shape of RDF.

Day 7 — JOIN: readings by role and equipment instead of UUID

Goal: combine `samples` and `lab_point` so you can ask for readings by meaning.

Read

A **JOIN** pairs rows from two tables that match a condition:

```
FROM samples AS s
JOIN lab_point AS p ON p.timeseries_id =
s.timeseries_id
```

For every reading `s`, find the point row `p` with the same `timeseries_id`, and glue them into one wider row. `s` and `p` are **aliases** (short names), so you can write `s.value` and `p.unit`.

- `JOIN` (inner join) keeps only rows that matched.
- `LEFT JOIN` keeps every row from the left table and fills the right side with NULLs when nothing matched. It's the tool for "which readings have no point?" style checks.

Once joined, you filter and group by point attributes: `WHERE p.openfdd_role = 'fan-status'`, `GROUP BY p.equipment_id`. No UUIDs typed.

Run

```
-- Supply fan run hours for every AHU, no
        UUIDs typed
SELECT p.equipment_id, round(sum(s.value)
        * 5 / 60.0, 1) AS run_hours
FROM samples AS s
JOIN lab_point AS p ON p.timeseries_id =
        s.timeseries_id
WHERE p.openfdd_role = 'fan-status'
GROUP BY p.equipment_id
ORDER BY p.equipment_id;
-- expect: AHU_1 | 1574.4


-- Average discharge vs return air
        temperature per AHU
SELECT p.equipment_id, p.openfdd_role,
        round(avg(s.value), 2) AS avg_f
FROM samples AS s
JOIN lab_point AS p ON p.timeseries_id =
        s.timeseries_id
WHERE p.openfdd_role IN ('discharge-air-
        temp', 'return-air-temp')
GROUP BY p.equipment_id, p.openfdd_role
ORDER BY p.equipment_id, p.openfdd_role;
-- expect: AHU_1 | discharge-air-temp |
        61.17
```

```
-- Integrity check: any readings whose
        UUID is not in the point list?
SELECT count(*)
FROM samples AS s
LEFT JOIN lab_point AS p ON
        p.timeseries_id =
        s.timeseries_id
WHERE p.timeseries_id IS NULL;
-- expect: 0
```

Zero orphans. The repo's tests enforce the same invariant on the RDF model: every series in the store has a point, and every point's series is in the store.

Try it

1. Average duct static pressure **setpoint** per AHU. Are the two AHUs controlled to the same setpoint?
2. Count readings per `unit`. Which unit has the most data?

Answers

```
SELECT p.equipment_id, round(avg(s.value),
        3) FROM samples s
JOIN lab_point p ON p.timeseries_id =
        s.timeseries_id
WHERE p.openfdd_role = 'duct-static-
        pressure-sp'
GROUP BY p.equipment_id ORDER BY
        p.equipment_id;
-- expect: AHU_1 | 1.366
```

AHU_1 averages 1.37 inH2O, AHU_2 0.78. Different setpoints, which is normal: each fan system is sized for its own ductwork.

```
SELECT coalesce(p.unit, '(none)') AS unit,
         count(*) AS readings
FROM samples s JOIN lab_point p ON
         p.timeseries_id =
         s.timeseries_id
GROUP BY unit ORDER BY readings DESC;
-- expect: DEG_F | 2380912
```

`coalesce(x, y)` returns the first non-NULL argument. Here it labels the points that have no unit (statuses, commands, alarms).

Why it matters

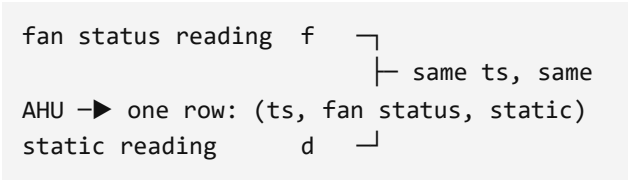
“Join readings to metadata, filter by metadata” is exactly what the API does. The difference is that its metadata comes from a SPARQL query over the graph rather than a table. Week 12’s client runs a SPARQL query, gets UUIDs back, and reads samples. Same join, across two systems.

Day 8 — Self-joins: static pressure *while the fan is running*

Goal: line up two different points at the same timestamp, and filter one by the other.

Read

In long format, the fan status and the duct static pressure at 10:05 are **two different rows** of `samples`. To ask "what was static pressure while the fan ran?", you need them side by side in one row. That takes a **self-join**: join `samples` to itself, once for each point, matching on the same `ts`.



Each copy of `samples` needs its own `lab_point` copy to pick the role:

```
FROM lab_point fp JOIN samples f  ON
f.timeseries_id = fp.timeseries_id
-- fan status readings
JOIN lab_point dp ON dp.equipment_id =
fp.equipment_id
                 AND dp.openfdd_role =
'duct-static-pressure'
-- same AHU's static point
JOIN samples d    ON d.timeseries_id =
dp.timeseries_id AND d.ts = f.ts
-- its reading at the same time
WHERE fp.openfdd_role = 'fan-status'
```

Matching on `ts` works because this historian logs every point on the same 5-minute grid. If two points were logged at different times, you'd first round them onto a common grid (Day 10).

Run

```
-- Duct static vs setpoint, only while the
        supply fan runs
SELECT fp.equipment_id,
       count(*)                      AS
         running_readings,
       round(avg(d.value), 3)        AS
         avg_static,
       round(avg(sp.value), 3)       AS
         avg_setpoint
FROM lab_point fp
JOIN samples f    ON f.timeseries_id =
        fp.timeseries_id
JOIN lab_point dp ON dp.equipment_id =
        fp.equipment_id AND
        dp.openfdd_role = 'duct-static-
        pressure'
JOIN samples d    ON d.timeseries_id =
        dp.timeseries_id AND d.ts = f.ts
JOIN lab_point spp ON spp.equipment_id =
        fp.equipment_id AND
        spp.openfdd_role = 'duct-static-
        pressure-sp'
JOIN samples sp   ON sp.timeseries_id =
        spp.timeseries_id AND sp.ts =
        f.ts
WHERE fp.openfdd_role = 'fan-status' AND
        f.value = 1
GROUP BY fp.equipment_id
ORDER BY fp.equipment_id;
-- expect: AHU_1 | 18893 | 1.347 | 1.402
```

Compare with Day 4. Over all hours AHU_1's static averaged 0.78 against a 1.37 setpoint, which looked like a fault. While running it's 1.35 against 1.40: close, slightly low. AHU_2

runs at **1.40 against a 0.76 setpoint**, well above. That's worth a look, and it's what a "duct static too high" rule would flag.

Try it

1. Average discharge air temperature on AHU_1 while the fan runs vs while it's off. Why is the minimum while running 0.0 °F?
2. Rewrite the first query for the fan **off** (`f.value = 0`). What's static pressure doing then?

Answers

```
SELECT f.value AS fan_status,
         round(avg(d.value), 1) AS
         avg_dat, round(min(d.value), 1)
         AS min_dat
FROM lab_point fp
JOIN samples f    ON f.timeseries_id =
         fp.timeseries_id
JOIN lab_point dp ON dp.equipment_id =
         fp.equipment_id AND
         dp.openfdd_role = 'discharge-
         air-temp'
JOIN samples d    ON d.timeseries_id =
         dp.timeseries_id AND d.ts = f.ts
WHERE fp.openfdd_role = 'fan-status' AND
         fp.equipment_id = 'AHU_1'
GROUP BY f.value ORDER BY f.value;
-- expect: 0 | 70.1 | 45.4
```

Running averages 53.3 °F, off 70.1 °F (the air in the duct drifts toward room temperature). A real discharge temperature of 0 °F is impossible. It's a dropout: the controller or historian wrote 0 when it had no value. Treat 0 in a temperature column as **missing**, not cold. Fault rules must not fire on it.

```
SELECT fp.equipment_id,
         round(avg(d.value), 3) AS
         avg_static_fan_off
FROM lab_point fp
JOIN samples f    ON f.timeseries_id =
         fp.timeseries_id
JOIN lab_point dp ON dp.equipment_id =
         fp.equipment_id AND
         dp.openfdd_role = 'duct-static-
         pressure'
JOIN samples d    ON d.timeseries_id =
         dp.timeseries_id AND d.ts = f.ts
WHERE fp.openfdd_role = 'fan-status' AND
         f.value = 0
GROUP BY fp.equipment_id ORDER BY
         fp.equipment_id;
-- expect rows: 2
```

Near zero, as it should be with the fan off. That's why the Day 4 averages were misleading.

Why it matters

Every fault rule is a condition on several points at the same time, gated by "is the equipment on?". Six joins to line up three

points is clumsy, though. On Day 11 you'll pivot the data once into a wide table and write rules as simple `WHERE` clauses.

Day 9 — CASE and conditional aggregation: run hours, percent occupied

Goal: compute several different counts in one pass using `CASE` inside aggregates.

Read

`CASE` is SQL's if/else, and it returns a value:

```
CASE WHEN condition THEN a
     WHEN other     THEN b
     ELSE c END
```

Put it inside an aggregate and you get **conditional aggregation**: one scan, many answers.

```
sum(CASE WHEN fan = 1 AND occ = 0 THEN 1
ELSE 0 END)   -- readings: running while
unoccupied
avg(CASE WHEN fan = 1 THEN static END)
-- average static only while running (ELSE
→ NULL, ignored)
```

In SQLite a comparison is already 0 or 1, so `sum(fan = 1 AND occ = 0)` works too. `CASE` is clearer and portable to other databases.

The `occupied` role is the AHU's occupancy command: 1 when the schedule says the building is in use. An AHU running while **unoccupied** is one of the most common energy findings in retro-commissioning (RCx).

Run

```
-- Hours each AHU ran while unoccupied,
         and was off while occupied
SELECT fp.equipment_id,
       round(sum(CASE WHEN f.value = 1 AND
         o.value = 0 THEN 1 ELSE 0 END) *
         5 / 60.0, 1) AS run_unocc_h,
       round(sum(CASE WHEN f.value = 0 AND
         o.value = 1 THEN 1 ELSE 0 END) *
         5 / 60.0, 1) AS off_occ_h,
       round(100.0 * avg(o.value), 1)
         AS pct_occupied
FROM lab_point fp
JOIN samples f    ON f.timeseries_id =
         fp.timeseries_id
JOIN lab_point op ON op.equipment_id =
         fp.equipment_id AND
         op.openfdd_role = 'occupied'
JOIN samples o    ON o.timeseries_id =
         op.timeseries_id AND o.ts = f.ts
WHERE fp.openfdd_role = 'fan-status'
GROUP BY fp.equipment_id
ORDER BY fp.equipment_id;
-- expect: AHU_1 | 58.6 | 2.2 | 51.3
```

AHU_1 ran about 59 hours while the schedule said unoccupied. AHU_2 has the opposite pattern: 30.8 hours **off** while occupied, which is a comfort problem rather than an energy one.

```
-- Bucket outside air into bands and count
SELECT CASE
         WHEN value < 40 THEN '1: below 40
         F'
         WHEN value < 60 THEN '2: 40-60 F'
         WHEN value < 80 THEN '3: 60-80 F'
         ELSE '4: 80 F and up'
       END AS band,
       count(*) AS readings
FROM samples
WHERE timeseries_id = 'bf422925-1e2d-598d-
         9069-3b0f17d49c48'
GROUP BY band
ORDER BY band;
-- expect rows: 4
```

Try it

Solve the week 1 challenge, now with `lab_point` and `CASE`: for each month, AHU_1's average outside air temperature, fan run hours, and readings with duct static above 1.5.

Answer

```
SELECT strftime('%Y-%m', s.ts,
         'unixepoch') AS month,
       round(avg(CASE WHEN p.openfdd_role
         = 'outside-air-temp' THEN
         s.value END), 1)            AS
         avg_oat,
       round(sum(CASE WHEN p.openfdd_role
         = 'fan-status' THEN s.value ELSE
         0 END) * 5 / 60.0, 1) AS run_h,
       sum(CASE WHEN p.openfdd_role =
         'duct-static-pressure' AND
         s.value > 1.5 THEN 1 ELSE 0 END)
         AS high_static
FROM samples s
JOIN lab_point p ON p.timeseries_id =
         s.timeseries_id
WHERE p.equipment_id = 'AHU_1'
  AND p.openfdd_role IN ('outside-air-
         temp', 'fan-status', 'duct-
         static-pressure')
GROUP BY month
ORDER BY month;
-- expect: 2026-03 | 41.5
```

One scan, three answers. The `CASE` picks which rows feed each column. In week 1 this took three queries, or a self-join.

Why it matters

A fault rule is a `CASE`: *when* the fan runs at full speed *and* static is below setpoint *then* 1 *else* 0. Sum it and you have fault hours. Day 14 builds exactly that.

Day 10 — Time buckets and views: hourly averages (what Timescale's `time_bucket` does)

Goal: downsample 5-minute data to hourly, and save queries as views and summary tables.

Read

Time bucketing. Integer division rounds a timestamp down to a bucket:

```
ts / 3600 * 3600     -- start of the hour (3600 s)
ts / 900  * 900      -- start of the 15-minute bucket
ts / 86400 * 86400   -- start of the UTC day
```

`ts` is an integer, so `/` drops the remainder. Group by the bucket and aggregate, and you have hourly data. TimescaleDB, the Postgres extension many historians use, calls this `time_bucket('1 hour', time)`. SQLite does it with arithmetic.

Always keep a `count(*)` per bucket. A bucket with 4 readings instead of 12 is a partial hour, and its average is less trustworthy.

Views. `CREATE VIEW name AS SELECT ...` saves a query under a name. It stores no data; it reruns every time you use it. Good for not repeating yourself.

Summary tables. `CREATE TABLE name AS SELECT ...` runs the query once and stores the result. It's fast to read, but stale when new data arrives. Timescale's **continuous aggregates** are this idea maintained for you: a summary table it refreshes as new data lands. In SQLite you rebuild it yourself.

Run

```
-- Hourly outside air temperature, AHU_1
SELECT datetime(ts / 3600 * 3600,
         'unixepoch') AS hour_utc,
       round(avg(value), 2) AS avg_f,
       count(*) AS n
FROM samples
WHERE timeseries_id = 'bf422925-1e2d-598d-
        9069-3b0f17d49c48'
GROUP BY ts / 3600
ORDER BY ts / 3600
LIMIT 2;
-- expect: 2026-03-16 00:00:00 | 69.03 | 4
```

The first hour has only 4 readings (00:40–00:55).

```sql
-- A view: every reading with its point
        attributes
DROP VIEW IF EXISTS lab_reading;
CREATE VIEW lab_reading AS
SELECT p.equipment_id, p.openfdd_role,
        p.point_id, p.unit, s.ts,
        s.value
FROM samples s
JOIN lab_point p ON p.timeseries_id =
        s.timeseries_id;

SELECT count(*) FROM lab_reading WHERE
        openfdd_role = 'fan-status';
-- expect: 71072


-- A summary table: hourly mean / min /
        max for every role point
DROP TABLE IF EXISTS lab_hourly;
CREATE TABLE lab_hourly AS
SELECT equipment_id, openfdd_role,
       ts / 3600 * 3600 AS hour_ts,
       avg(value) AS mean, min(value) AS
         lo, max(value) AS hi, count(*)
         AS n
FROM lab_reading
WHERE openfdd_role IS NOT NULL
GROUP BY equipment_id, openfdd_role, ts /
        3600;

SELECT count(*) FROM lab_hourly;
-- expect: 77012
```

26 role points × 2,962 hours = 77,012 rows, instead of 924k readings. Charts and dashboards read tables like this.

Try it

1. From `lab_hourly`, the hour with AHU_2's highest average duct static pressure.
2. Write a 15-minute bucket version of the first query. How many readings does a full 15-minute bucket have?

Answers

```
SELECT datetime(hour_ts, 'unixepoch'),
         round(mean, 3)
FROM lab_hourly
WHERE equipment_id = 'AHU_2' AND
         openfdd_role = 'duct-static-
         pressure'
ORDER BY mean DESC
LIMIT 1;
-- expect rows: 1
```

```
SELECT datetime(ts / 900 * 900,
         'unixepoch') AS bucket, count(*)
         AS n
FROM samples
WHERE timeseries_id = 'bf422925-1e2d-598d-
         9069-3b0f17d49c48'
GROUP BY ts / 900 ORDER BY ts / 900 LIMIT
         3;
-- expect: 2026-03-16 00:30:00 | 1
```

A full 15-minute bucket has 3 readings. The first has only one, at 00:40.

Week 2 challenge

Build `lab_daily`: one row per AHU per day, with fan run hours, hours run while unoccupied, average outside air temperature, and average static pressure *while running*. Then find the five days with the most unoccupied runtime on AHU_1, and check the day of week (`strftime('%w', ...)`): weekends?

Reading — Week 2

The lessons first, then these for exact behavior.

- SQLite joins (SELECT, the FROM clause)
- SQLite CASE expressions
- SQLite CREATE VIEW
- DB Browser: importing CSV
- TimescaleDB time_bucket